

The University of Melbourne
 COMP30024 – Artificial Intelligence
 Project Part B: Game Playing Agent

Playing Cascade Using Minimax

Water Bottle
 Phuong Trang Tran – 1409465
 Ha Linh Nguyen – 1482069
 May 18, 2026

Overview

This report describes the design and implementation of a game-playing agent for Cascade, a two-player adversarial board game. The agent is built on iterative deepening alpha-beta minimax with a multi-feature evaluation function, a transposition table, and dedicated mechanisms for handling draw-by-repetition. The report is structured as follows: Section 1 describes the final agent’s action selection approach; Section 2 evaluates its performance; Section 3 documents failed heuristic trials that motivated the final design; Section 4 covers technical optimisations; and Section 5 describes supporting development work.

1. Approach: Action Selection

1.1 Overview

The agent selects actions using **iterative deepening alpha-beta minimax** as its core search strategy. On every turn, the algorithm performs successive full-depth searches from depth 1 up to a maximum of 12, subject to a per-move time budget of 2.0 seconds. The search is guided by a multi-feature evaluation function, a transposition table, and a move ordering system. Two dedicated mechanisms handle draw-by-repetition, which was identified as the dominant failure mode in earlier versions.

1.2 Search Algorithm

Iterative Deepening. At each turn, the agent runs alpha-beta minimax at depth 1, then depth 2, and so on, until the time budget is exhausted. If a `TimeoutError` is raised mid-depth, the result from the last fully completed depth is committed. This ensures the agent always returns a well-reasoned move even under time pressure, while naturally using as much of the available time as possible.

Alpha-Beta Pruning. Alpha-beta pruning is applied throughout the search tree. When a beta-cutoff occurs — a move is proven too good for the maximising player, so the minimising player would avoid the subtree — the remaining moves at that node are skipped. For well-ordered move lists, this reduces the effective branching factor to approximately the square root of the full tree, allowing the agent to search roughly twice as deep as plain minimax in the same time.

Transposition Table. A transposition table (TT) is maintained across the entire game, not just a single turn, so search results from earlier turns can benefit later ones. Each entry stores a score, a bound type, and the depth at which it was computed:

- `TT_EXACT`: the stored score is exact — return it directly.
- `TT_LOWER` (alpha bound): use it to raise alpha.
- `TT_UPPER` (beta bound): use it to lower beta.

The table is capped at 500,000 entries to prevent unbounded memory growth.

1.3 Move Ordering

Quality move ordering is critical to alpha-beta efficiency. A `MoveOrderer` class maintains two complementary heuristics across the entire game:

Killer Moves. Up to two moves per search depth that have previously caused a beta-cutoff are stored.

These are tried before other moves at the same depth in subsequent iterations, as they are statistically likely to cause cutoffs again.

History Heuristic. Every move that causes a cutoff increments a persistent counter weighted by 2^{depth} . Moves with higher history scores are searched earlier. This rewards moves that are consistently strong across different board positions throughout the game.

Static Priority. Within each node, moves are additionally ordered by static type before killer and history scores are applied:

1. Winning moves (captures the opponent's last token) — searched first.
2. `EatAction` moves, ordered by tokens captured (larger captures first).
3. `CascadeAction` moves that threaten opponent tokens.
4. `MoveAction` moves.
5. `PlaceAction` moves, ordered by proximity to the board centre.

1.4 Repetition and Cycle Handling

Path-Set Cycle Detection. A mutable set of board hashes (`path_set`) is threaded through every level of the minimax recursion. Before expanding a node, the algorithm checks whether the current board hash is already in the set — if so, the line leads to a draw by repetition and is scored 0. The hash is added before recursing and removed (`discard`) after, maintaining the invariant that the set contains exactly the positions on the current search path.

Game History Penalty at Leaf Nodes. The actual game's `position_history` dictionary (mapping board hash to visit count) is passed into minimax as a read-only parameter. At depth-0 leaf nodes, positions that have already been visited in the real game receive a penalty of -150 per prior visit. This steers the agent away from revisiting real-game positions and converts oscillation patterns into decisive lines.

Threefold Repetition Avoidance. At the root of `get_best_move`, any move whose resulting position hash already appears two or more times in `position_history` is scored as a draw (-5000 if a better alternative exists, otherwise 0), preventing an automatic loss-by-draw.

1.5 Evaluation Function

The heuristic estimates board quality from the agent's perspective using six additive terms:

$$h(s) = 100(T_{\text{self}} - T_{\text{opp}}) + 1.5(H_{\text{self}}^2 - H_{\text{opp}}^2) + 2.0(E_{\text{opp}} - E_{\text{self}}) + 20(A_{\text{self}} - A_{\text{opp}}) + 30(C_{\text{self}} - C_{\text{opp}}) \quad (1)$$

where T denotes total tokens, H^2 denotes sum of squared stack heights, E denotes edge penalty (height-weighted proximity to board edge), A denotes immediate eat-threat tokens, and C denotes cascade-threat tokens.

Material Advantage. The difference in total token count is given the dominant weight of 100, reflecting that token elimination is the primary winning condition of Cascade.

Height Advantage. Stack heights are squared and summed for each side. The squared term rewards concentration, a single tall stack is worth more than two short ones of equivalent total height, reflecting the greater cascade potential of taller stacks.

Edge Penalty Advantage. Tokens near the board edge have fewer available moves and are more vulnerable to being pushed off by cascade actions. The net edge penalty rewards the agent for keeping its own tokens central while pushing the opponent toward the edges.

Eat Threats. The net number of tokens immediately capturable by each side (via `EatAction`) is weighted at 20 per token. This is a strong short-term tactical signal: being able to capture while avoiding capture is directly decisive.

Cascade Threats. For each tall stack (height ≥ 3), the search checks whether a cascade in any direction would hit an opponent token. The net cascade-threat advantage is weighted at 30 per token, higher than simple eat threats, reflecting the chained destructive potential of cascade actions.

Centre Bonus (Placement Phase Only). During the four-token placement phase (`_turn_count < 8`), each own token receives a bonus of $(7 - d_{\text{centre}}) \times 10$, rewarding central positions that maximise future

reach.

1.6 Conclusion

The agent’s action selection combines iterative deepening alpha-beta search with a transposition table, killer move and history heuristic ordering, path-set cycle detection, and a six-term evaluation function. Together these components address the three main challenges of Cascade: tactical capture opportunities, positional stability, and avoidance of draw-by-repetition.

2. Performance Evaluation

2.1 Overview

The agent’s performance was evaluated through direct match-play against three reference opponents of increasing strength: a random agent, a greedy one-ply agent, and an alpha-beta minimax reference agent at fixed depth 3 with no transposition table. For each opponent, 20 games were played (10 as RED and 10 as BLUE), and win, draw, and loss counts were recorded.

2.2 Results

Table 1 summarises the results across all 60 test games (20 per opponent, 10 as each colour). The agent won 59 of 60 games, with one draw as BLUE against the depth-3 minimax reference and no losses against any opponent.

Opponent	Games	Wins	Draws	Losses	Win Rate
Random agent	20	20	0	0	100%
Greedy agent	20	20	0	0	100%
Minimax (depth 3)	20	19	1	0	95%
Total	60	59	1	0	98.3%

Table 1: Match results across all 60 test games (20 per opponent; 10 as RED, 10 as BLUE). The single draw occurred as BLUE against the minimax reference.

2.3 Identified Failure Modes

Draw-by-Repetition. In early versions, the agent frequently oscillated between positions, resulting in drawn games that should have been decisive wins. This was the dominant failure mode and directly motivated the path-set cycle detection and game history penalty mechanisms described in Section 1.

Shallow Search Depth. Before the `position.history.copy()` performance fix (Section 4), the agent could rarely exceed depth 4 within the time budget. After the fix, it reliably reached depth 8–10+ in the mid-game, substantially improving the quality of moves in critical positions.

2.4 Conclusion

The combination of deeper search and robust repetition handling was the decisive factor in moving the agent from a draw-prone, shallow-searching player to one that wins the clear majority of games against the depth-3 minimax reference. Remaining losses are predominantly attributable to time-pressure situations in which the search terminates at a lower depth than usual.

3. Failed Trials

3.1 Overview

Three agent designs were trialled and discarded before arriving at the final evaluation function. Each failed to produce decisive play against minimax-based opponents, though each failure revealed a specific design principle that informed the next iteration.

3.2 Trial 1: Starting Agent — Pure Token-Difference Minimax

3.2.1 Design Rationale

The first implementation was a minimal alpha-beta minimax agent built to establish a working baseline. The evaluation function was a single-term token difference:

$$h(s) = T_{\text{self}} - T_{\text{opp}} \quad (2)$$

No weights, positional terms, or tactical features were included. The search ran at fixed depth 3 with no move ordering, and the board was stored as a nested 8×8 list copied with `copy.deepcopy` at every node.

3.2.2 Observed Behaviour

The agent produced legal, coherent play and won reliably against random opponents. Against the greedy and depth-3 minimax reference agents it drew or lost frequently.

3.2.3 Identified Limitations

- **No positional awareness:** stacks were regularly placed on edges, where they were vulnerable to cascade push-off.
- **No tactical signals:** immediate eat threats and cascade opportunities were not rewarded.
- **No repetition penalty:** the agent entered oscillation loops against stronger opponents.
- **Fixed depth, no move ordering:** alpha-beta pruning was inefficient; search depth could not adapt to available time.

3.3 Trial 2: Revised Heuristic with Tactical Penalty

3.3.1 Design Rationale

After testing Trial 1, the heuristic was revised to penalise stacks vulnerable to immediate capture, increase the material weight, and reduce the height reward unless it contributed to concrete tactical opportunities. This made the agent more stable but less aggressive against minimax-based opponents.

3.3.2 Observed Behaviour

The revised agent showed improved resistance to immediate capture threats and won more consistently against simple opponents. Against minimax-based agents the draw rate remained high.

3.3.3 Identified Limitations

Non-functional Repetition Penalty. The repetition penalty was structurally broken: `GameState.copy()` resets `position_history` to an empty dictionary, so every search node has no memory of prior positions. The penalty evaluated to zero throughout the tree.

Incomplete Board Hash. The board hash omitted the current player’s turn colour, conflating strategically distinct positions and further undermining the repetition penalty.

3.4 Trial 3: Intermediate Agent — Expanded Heuristic with Static Move Ordering

3.4.1 Design Rationale

Building on Trials 1 and 2, a substantially revised agent was implemented as a direct predecessor to the final version. The evaluation function was extended to four terms:

$$h(s) = 50 (T_{\text{self}} - T_{\text{opp}}) + w_c \sum_{\text{own}} (-d_{\text{centre}} \cdot h) + 20 N_{\text{eat}} - 500 R \quad (3)$$

where $w_c = 25$ during placement and 1 otherwise, N_{eat} counts own tokens immediately capturable adjacent to opponent stacks, and R is the current-position visit count. A static move-ordering function (`EatAction` > `CascadeAction` > `MoveAction`) was added, and the search depth was made phase-adaptive (depth 1 during placement, depth 3 during play).

3.4.2 Observed Behaviour

The agent placed pieces centrally, prioritised eat actions, and won consistently against random and greedy opponents. Against the depth-3 minimax reference agent, drawn games remained frequent.

3.4.3 Identified Limitations

- **Broken repetition penalty:** the same `GameState.copy()` issue as Trial 2; the penalty evaluated to zero at every search node.
- **No iterative deepening:** fixed depths could not adapt to available time, risking timeout in complex positions.
- **No transposition table:** identical positions reached via different move sequences were re-evaluated from scratch.
- **deepcopy overhead:** the nested-list board made state copy the dominant runtime cost, capping practical search depth.
- **Static move ordering:** fixed type-based priority did not learn from cutoffs; killer moves and history heuristic were absent.

Trial 3 directly motivated the key architectural changes in the final agent: iterative deepening with time management, a persistent transposition table, the `MoveOrderer` class with killer moves and history heuristic, path-set cycle detection, and a flat-array board representation.

4. Other Technical Aspects

4.1 Overview

Several algorithmic and data-structure optimisations were implemented to maximise the agent's search depth within the per-move time budget. These improvements operate below the level of the search algorithm and evaluation function but have a significant effect on practical performance.

4.2 State Representation

Flat Board Array. `GameState` stores the board as a flat `list[64]` rather than a nested 2D array. This makes `state.copy()` a single list slice (`board[:]`), one of the most performance-critical operations, as a copy is made for every node in the search tree.

Incremental Token Counts. `red_tokens` and `blue_tokens` are updated incrementally every time a cell is written via `set()`. Terminal detection and the material advantage term in the heuristic therefore require no board scan and so they execute in $O(1)$.

Lazy Board Hash. `board_hash()` uses a dirty flag (`_hash_dirty`). The hash is recomputed only when the board has changed since the last call; multiple calls on an unchanged state return the cached result. The hash is a tuple of (`cell_index`, `color`, `height`) for all occupied cells, directly usable as a dictionary key in the TT and `path_set`.

4.3 Precomputed Lookup Tables

Adjacency Table. `_ADJ[r][c][direction]` is a module-level table containing the (`nr`, `nc`) neighbour of every cell in every direction, or `None` if out of bounds. This eliminates repeated bounds-checking arithmetic in move generation, heuristic evaluation, and move ordering, all of which iterate over neighbours in their innermost loops.

Centre Distance Table. `_CENTRE_DIST[r * 8 + c]` precomputes the Manhattan distance from the board centre (3.5, 3.5) for all 64 cells. This is used in the placement heuristic and move ordering without any arithmetic at runtime.

4.4 Iterative Deepening Re-ordering

After each completed depth, the best move found is placed at the front of the move list for the next iteration. This implements a lightweight form of principal variation re-ordering without maintaining a full PV table. The best move is then tried first at the next depth, maximising its contribution to alpha-beta pruning.

4.5 Conclusion

The combination of a flat board representation, incremental token counts, a lazy hash, and precomputed adjacency and distance tables substantially reduces per-node overhead. These optimisations were the primary enabler of the deeper search depths that distinguish the final agent’s performance from earlier versions.

5. Supporting Work

5.1 Overview

Development of the agent was supported by systematic match-play testing against agents of varying strength, and by computational complexity analysis to identify and address performance bottlenecks. Both activities provided direct evidence of failure modes and guided each iterative improvement.

5.2 Testing Methodology

The agent was tested progressively against three opponents:

- a **random agent**, to verify basic correctness of move generation and search,
- a **greedy one-ply agent**, to verify that the search was exploiting simple tactical opportunities, and
- the **minimax reference agent** at fixed depth 3, as the primary benchmark for final performance.

Match results were observed to identify specific failure modes. The two most impactful findings, draw-by-repetition dominance and the `position.history.copy()` allocation bottleneck, were discovered through this testing process and directly produced the path-set cycle detection and game history penalty mechanisms.

5.3 Complexity Analysis

Big- O analysis was used alongside testing to identify the most time-consuming operations in the search loop and helps us in optimising the algorithm. This analysis identified per-node memory allocation (specifically `position.history.copy()`) as the dominant overhead, and informed the decision to use the read-only `game.history` parameter pattern instead. The time budget per move was calibrated based on an estimate of expected move count across a full game (approximately 150 play turns), dividing the 180-second total budget accordingly.

5.4 Iterative Comparison

Intermediate agent versions were compared by running back-to-back match series and observing win/draw/loss distributions. Individual changes like heuristic weight adjustments, transposition table integration, move ordering additions were each evaluated in isolation before being committed to the final version, ensuring that improvements were additive and regressions were caught early.

5.5 Conclusion

Direct match-play testing against a range of opponents, combined with complexity analysis, was the primary tool for diagnosing weaknesses and validating improvements. This approach enabled targeted, evidence-driven development and was more informative than static code analysis alone, as several key failure modes only became apparent through observed game behaviour.

References

- [1] GeeksForGeeks. (n.d.). *Minimax algorithm in game theory — Set 1 (introduction)*. <https://www.geeksforgeeks.org/dsa/minimax-algorithm-in-game-theory-set-1-introduction/>
- [2] Hoin, T. K. (2016). *AI-Project-B* [Computer software]. GitHub. <https://github.com/tuankhoin/AI-Project-B>
- [3] Farrugia, M. (2020). *AI-projectB* [Computer software]. GitHub. <https://github.com/matomatical/AI-projectB>
- [4] Anthropic. (2026). *Claude* [Large language model]. <https://claude.ai>
- [5] OpenAI. (2026). *ChatGPT* [Large language model]. <https://chat.openai.com>